

ADT

Table of contents

First-Class ADT Pattern	2
First-Class ADT Pattern	2
Encapsulation	2
Abstract Data Type (ADT)	2
ADT in C e in Python	3
First-Class ADT	3
ADT: teoria vs C (con e senza incapsulamento) vs Python	4
STACK	5
Operazioni essenziali	6
Operazioni complementari	6
Operazioni di gestione (tipiche in C)	7
Implementazione dello Stack con Array Statico	7
Scelte progettuali	8
Responsabilità del <code>main</code>	8
Dimensione massima dello stack	8
Assenza di incapsulamento completo	9
Uso dei puntatori	9
Altre note	9
Esempio di <code>main</code>	10
<code>module.h</code>	10
<code>module.c</code>	12
Discussione e miglioramenti	15
Verifica condizioni di overflow/underflow	15
Stack con capacità dinamica	15
Dangling pointer	17
Stack con Array Statico, incapsulamento completo	18
<code>module.h</code>	18
<code>module.c</code>	19
Stack using Linked List	26
Stack using a Linked List- only abstraction	29
Stack using a Linked List - First Class ADT	29

QUEUE - (First In First Out	IN PROGRESS	31
put (enqueue) and get (dequeue)		33
DYNAMIC QUEUE (LINKED LIST) (1)		35
DYNAMIC QUEUE (LINKED LIST) (2)		39
QUEUE (using an array)		39
A possible solution		40
Exercises		44
FONTI PRINCIPALI		45

First-Class ADT Pattern

First-Class ADT Pattern

Encapsulation

L'incapsulamento (**encapsulation**) è uno strumento fondamentale per gestire la complessità e separare le responsabilità all'interno di un sistema.

Quando si progetta software, è utile distinguere tra due prospettive:

- **Comportamento esterno (interfaccia):** come un componente viene utilizzato (come interagisce con l'esterno)
- **Implementazione interna:** come il componente è effettivamente costruito (come è strutturato internamente)

Abstract Data Type (ADT)

Un **Tipo di Dato Astratto (ADT)** definisce:

- un **insieme di valori**
- un **insieme di operazioni** su tali valori

senza specificare come questi valori siano rappresentati internamente.

https://en.wikipedia.org/wiki/Abstract_data_type

Vedi anche [Code Complete / Steve McConnell.-2nd ed.]. Un estratto del Cap. 6 è disponibile insieme alle slides.¹

¹“Il riassunto, la citazione o la riproduzione di brani o di parti di opera e la loro comunicazione al pubblico sono liberi se effettuati per uso di critica o di discussione, nei limiti giustificati da tali fini e purché non costituiscano concorrenza all'utilizzazione economica dell'opera; se effettuati a fini di insegnamento o di ricerca scientifica l'utilizzo deve inoltre avvenire per finalità illustrative e per fini non commerciali.” - <https://www.normattiva.it/uri-res/N2Ls?urn:nir:stato:legge:1941-04-22;633~art70>

ADT in C e in Python

Nel linguaggio C, l'incapsulamento di un ADT si realizza tipicamente tramite:

- la dichiarazione di un **tipo incompleto** (*incomplete type*, dato opaco) nel file header `.h`
- la manipolazione dei valori esclusivamente tramite **puntatori**
- la definizione completa della struttura nascosta nel file `.c`

Il tipo è detto *incompleto* perché la sua struttura interna non è visibile al codice che utilizza l'ADT.

In **Python**, gli ADT si implementano tipicamente tramite **classi**, che nascondono (parzialmente) i dettagli interni e forniscono un'interfaccia pubblica.

First-Class ADT

Un ADT è detto **first-class** se i suoi valori possono essere trattati come qualsiasi altro valore del linguaggio.

https://en.wikipedia.org/wiki/First-class_citizen

In C, questo significa che (tipicamente tramite puntatori) un ADT può:

- essere **passato come argomento a una funzione**
- essere **restituito da una funzione**
- essere **assegnato a una variabile**
- essere **memorizzato in strutture dati**

In C, gli ADT sono generalmente gestiti tramite puntatori: l'assegnazione tra variabili copia il puntatore (riferimento), non il contenuto dell'oggetto.

Nel contesto della programmazione, si parla di “**cittadini di prima classe**” (**first-class citizens**) per indicare entità che possono essere usate liberamente: assegnate a variabili, passate come argomenti, restituite da funzioni e memorizzate in strutture dati. In **C**, sono first-class i valori “semplici” (int, float, puntatori, ecc.), mentre le **funzioni non lo sono pienamente**: non possono essere assegnate direttamente a variabili, ma solo tramite **puntatori a funzione**. In **Python**, invece, anche le funzioni sono veri cittadini di prima classe.

```
// Esempio in C
int add(int a, int b) { return a + b; }

int main() {
    int (*f)(int, int) = add; // puntatore a funzione
    int r = f(2, 3);          // uso indiretto
}
```

```
def f1(x):
    print(x)
    #return None
def f2(x):
    print(x)

a = f2 # assegnazione diretta

f1(a) # <function f2 at 0x110211ee0>
```

-
- **ADT con incapsulamento completo dei dati.**
L'attenzione è su **astrazione** e **incapsulamento**: la rappresentazione interna è nascosta e accessibile solo tramite le operazioni definite.
(È un'implementazione **corretta e robusta** di un ADT).
 - **ADT senza incapsulamento completo dei dati.**
L'attenzione è solo sull'**astrazione**: la struttura interna è visibile o direttamente accessibile.
(È un'implementazione **debole** di un ADT, che viola l'incapsulamento).
 - Un ADT può inoltre essere *first-class* se i suoi valori possono essere passati, restituiti e assegnati come qualsiasi altro valore del linguaggio.

ADT: teoria vs C (con e senza incapsulamento) vs Python

Aspetto	Teoria (ADT)	C (con incapsulamento)	C (senza incapsulamento)	Python
Definizione	Insieme di valori + operazioni	Uguale alla teoria	Uguale alla teoria	Uguale alla teoria
Rappresentazione interna	Non specificata	Nascosta (struct in .c)	Visibile (struct in .h)	Attributi della classe

Aspetto	Teoria (ADT)	C (con incapsulamento)	C (senza incapsulamento)	Python
Incapsulamento	Non richiesto	Forte (tipo incompleto)	Assente	Debole (convenzioni)
Accesso ai dati	Tramite operazioni	Solo tramite funzioni	Diretto + funzioni	Diretto + metodi
Modifica dei dati	Tramite operazioni	Tramite funzioni	Diretto + funzioni	Diretto + metodi
First-class	Dipende dal linguaggio	Tramite puntatori	Tramite puntatori	Nativamente

STACK

[https://en.wikipedia.org/wiki/Stack_\(abstract_data_type\)](https://en.wikipedia.org/wiki/Stack_(abstract_data_type))

Lo **stack** (o **pila**) è un ADT in cui gli elementi vengono gestiti secondo la politica **LIFO** (*Last In, First Out*): l'ultimo elemento inserito è il primo a uscire.

Le operazioni fondamentali sono:

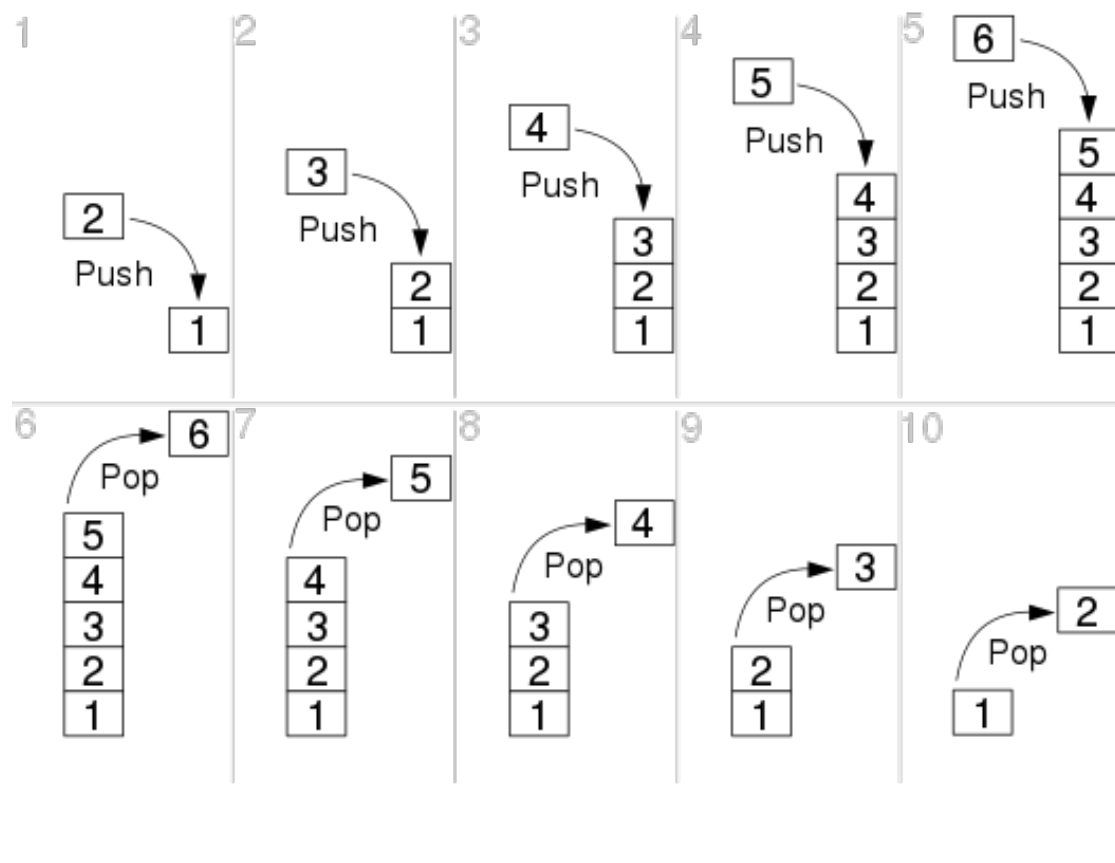
- **push**: inserisce un nuovo elemento in cima allo stack
- **pop**: rimuove l'elemento in cima allo stack
- **peek**: restituisce l'elemento in cima senza rimuoverlo

Tutte queste operazioni agiscono su un'unica estremità della struttura, detta **cima** (*top*).

In C, lo stack può essere implementato con:

- un **array**
- una **lista concatenata**

La scelta dell'implementazione influenza gli aspetti pratici (uso della memoria, dimensione massima, efficienza), ma non modifica il comportamento astratto dello stack. Uno stack è definito dalle sue operazioni e dal suo comportamento LIFO, non dal modo in cui viene implementato.



Operazioni essenziali

Sono le operazioni **minime** che definiscono il comportamento dello stack:

- `push(Stack* s, T x)`
Inserisce un elemento in cima allo stack
- `pop(Stack* s)`
Rimuove l'elemento in cima allo stack

Operazioni complementari

Non sono strettamente necessarie per definire l'ADT, ma sono **molto utili** nella pratica:

- `peek(Stack* s) / peek(...)`
Restituisce l'elemento in cima senza rimuoverlo
- `isEmpty(Stack* s)`
Verifica se lo stack è vuoto

- `size(Stack* s)`
Restituisce il numero di elementi

Operazioni di gestione (tipiche in C)

In C sono fondamentali per gestire memoria e ciclo di vita:

- `create() / init()`
Crea e inizializza uno stack
- `destroy(Stack* s)`
Libera la memoria associata
- `isFull(Stack* s)`
non è un'operazione propria dello stack come ADT, ma può essere introdotta in implementazioni con capacità limitata.

Implementazione dello Stack con Array Statico

Uno stack può essere implementato utilizzando:

- un **array statico** per memorizzare gli elementi
- una variabile intera **top** che indica la posizione dell'elemento in cima allo stack

```
typedef struct stack {  
    int stackArray[N];  
    int top;  
} Stack;
```

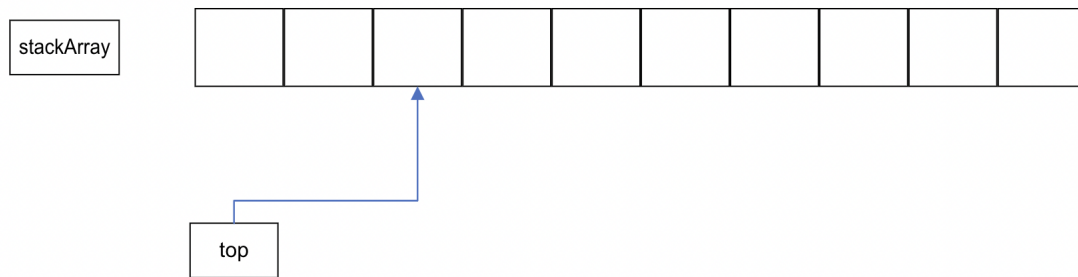
Quando lo stack è vuoto, **top** vale:

```
top = -1;
```

Le operazioni di inserimento o estrazione modificano **top**:

- **push** incrementa **top** e inserisce un elemento
- **pop** restituisce l'elemento in cima e decrementa **top**

In questo modo, **top** rappresenta sempre l'**indice dell'ultimo elemento** inserito nello stack.



Scelte progettuali

Questa versione è **semplificata e intenzionalmente non completa** dal punto di vista progettuale.

Responsabilità del main

In questa implementazione, il controllo delle condizioni di corretto utilizzo dello stack è delegato al programma chiamante (**main**).

In particolare:

- prima di eseguire una **pop** o **peek**, il **main** deve verificare che lo stack **non sia vuoto** mediante la funzione `isEmpty()`
- prima di eseguire una **push**, il **main** deve verificare che lo stack **non sia pieno** mediante la funzione `isFull()`

Questa è una **scelta didattica**:

le funzioni del modulo non effettuano controlli interni e richiedono il rispetto di precise **precondizioni**.

Dimensione massima dello stack

La dimensione dello stack è fissata a compile-time tramite una direttiva:

```
#define STACK_MAX_SIZE 100
```

Questa costante è definita nel file di interfaccia (**.h**) ed è quindi **visibile a tutto il programma**. Si tratta di una scelta che privilegia semplicità e chiarezza del modello a scapito della flessibilità.

Assenza di incapsulamento completo

La struttura è visibile nell'interfaccia `.h`. Il main può bypassare le funzioni di interfaccia.

```
s->stackArray[0]=10;
s->top=0;
```

Uso dei puntatori

Le funzioni operano su **puntatori a Stack**. Questa scelta è motivata da **efficienza**, perché si evita la copia della struttura, e **coerenza progettuale**, perché facilita l'evoluzione verso implementazioni più avanzate, per esempio incapsulamento completo. Tuttavia questo non è un obbligo. Dato che la struttura è visibile nell'interfaccia, il main può dichiarare direttamente variabili `struct stack` e manipolare i campi.

Altre note

Questa implementazione presenta alcune limitazioni intenzionali:

- non verifica condizioni di errore (overflow/underflow) nelle funzioni `push()`, `pop()`, `peek()`. Il controllo è demandato al main, che prima di chiamare `push()` deve verificare, mediante `isFull()`, che lo stack abbia ancora capienza, e prima di chiamare `pop()` e `peek()` deve verificare, mediante `isEmpty()`, che lo stack non sia vuoto.
- espone direttamente la struttura dati
- utilizza una dimensione fissa per l'array che memorizza i dati dello stack

Tali scelte la rendono semplice da comprendere e adatta a una prima introduzione, ma spesso non adeguata a contesti reali. Viene fornito un main con un menu ed un test minimali, e una funzione `show()` che mostra tutti gli elementi dello stack.

Le funzioni di debug come `show()` e `automaticTest()` sono **solo per debug**. Una volta terminato il debug dovrebbero essere rimosse dall'interfaccia (o rimosse del tutto), perché non fanno parte dell'ADT, e dichiarate `static` nel `modulo.c`. Togliere la dichiarazione di una funzione dal file `.h` è generalmente sufficiente per impedirne l'uso da `main.c`, perché il compilatore non ne vede la dichiarazione. Tuttavia, questo non è completamente sicuro: se qualcuno dichiara manualmente `void show(Stack *);` in `main.c`, il programma può comunque compilare e linkare, perché la funzione esiste in `module.c`. Per evitare questo "bypass", è buona pratica dichiarare `show()` e `automaticTest()` come `static` nel `.c`: in questo modo la funzione è visibile solo all'interno del file e non può essere usata dall'esterno.

Esempio di main

(Nel repository github il main completo)

```
int main() {
    int value = 0;
    Stack *s = initStack();

    // Push (with check)
    if (!isFull(s)) {
        push(s, value);
    } else {
        printf("Stack pieno!\n");
    }

    // Peek (with check)
    if (!isEmpty(s)) {
        value = peek(s);
    } else {
        printf("Stack vuoto!\n");
    }

    // Pop (with check)
    if (!isEmpty(s)) {
        value = pop(s);
    } else {
        printf("Stack vuoto!\n");
    }

    // ora lo stack è vuoto

    push(s, 1);
    push(s, 2);
    push(s, 3);

    show(s);
}
```

module.h

```

// ADT/stack_array_noencaps/module.h

/* STACK
* Implementazione: array statico
* SENZA incapsulamento completo
*
* È possibile aggirare l'interfaccia e violare l'incapsulamento:
* s->stackArray[0] = 10;
* s->top = 0;
*
* L'implementazione utilizza un array contenuto in una struttura.
* L'allocazione della struttura è dinamica, ma questo è solo un dettaglio implementativo.
* Modificando initStack si potrebbe ottenere un'implementazione completamente statica.
*/

#ifdef MODULE_H
#define MODULE_H
#include <stdbool.h>

/*
* il .h deve includere altri .h solo se strettamente necessario.
* La libreria <stdbool.h> definisce
*   il tipo di dato booleano bool
*   le costanti true (1) e false (0).
* Viene usato dall'interfaccia, quindi è necessario includerlo
*/

/* STACK_MAX_SIZE -> capacity of the Static Stack
* STACK_MAX_SIZE is part of the interface
* and it is visible from main()
*/
#define STACK_MAX_SIZE 1000

typedef struct stack {
    int stackArray[STACK_MAX_SIZE];
    int top;
}Stack;

// Function prototypes

// Push element to the top of the stack
// PRECONDIZIONE: stack NON pieno
void push(Stack * stack, int val);

```

```

// Remove and return the top most element of the stack
// PRECONDIZIONE: stack NON vuoto
int pop(Stack * stack);

// Return the top most element of the stack
// PRECONDIZIONE: stack NON vuoto
int peek(Stack * stack);

// Check if the stack is in Underflow state or not
bool isEmpty(Stack * stack);

Stack * initStack(); //INIT
void deleteStack(Stack * stack); // Free memory

// Check if the stack is in Overflow state or not
// ONLY FOR STATIC ARRAY IMPLEMENTATION
bool isFull(Stack * stack);

// FOR DEBUG USE ONLY - REMOVE FROM THE INTERFACE
void show(Stack * stack);
void automaticTest(void);

#endif //MODULE_H

```

module.c

```

// ADT/stack_array_noencaps/module.c

#include <stdlib.h>
#include <stdio.h>
#include <assert.h>
#include "module.h"

Stack * initStack(){
    Stack * stack= malloc(sizeof(*stack));
    if (!stack) {
        printf("Memory allocation failed!\n");
        exit(EXIT_FAILURE); // Stop the program if malloc fails
    }
}

```

```

        stack->top=-1;
        return stack;
};

void deleteStack(Stack * p_s){
    // attenzione, dangling pointer
    // il chiamante mantiene un puntatore dangling se non lo azzera con p=NULL
    free (p_s);
}

void push(Stack * stack, int val){

    stack->top+=1;
    stack->stackArray[stack->top] = val;
}

// PRECONDIZIONE: stack NON vuoto
int pop(Stack * stack){
    int val = stack->stackArray[stack->top];
    stack->top-=1;
    return val;
}

// PRECONDIZIONE: stack NON vuoto
int peek(Stack * stack){
    int val = stack->stackArray[stack->top];
    return val;
}

bool isEmpty(Stack * stack){ return stack->top== -1; }

bool isFull(Stack * stack){ return stack->top == STACK_MAX_SIZE-1; }

// FOR DEBUG USE ONLY - REMOVE FROM THE INTERFACE
void show(Stack * stack){
    int i;
    if (isEmpty( stack)) {
        printf("Empty stack!\n");
        return;
    }
    printf("[TOP] [");
    for (i=stack->top; i>=0; i--)
        printf("%d ",stack->stackArray[i]);
    printf("] [BASE]\n");
}

```

```

void automaticTest(void) {
    int value;
    Stack * s;
    s = initStack();
    printf("Automatic test\n");
    printf(" - check empty\n");
    assert(isEmpty(s)); //https://en.wikipedia.org/wiki/Assert.h
    printf(" - check push, pop, peek\n");
    // Push 1
    value=1;
    push(s, value);
    assert( !(isEmpty(s)) );
    assert(value == peek(s));
    assert( !(isEmpty(s)) );
    assert(value == pop(s));
    assert((isEmpty(s)));

    // Push 1, 2, 3
    push(s, 1);
    push(s, 2);
    push(s, 3);
    printf("->expected: \n[TOP] [3 2 1] [BASE]\n");
    printf("->obtained:\n");
    show(s);
    assert( !(isEmpty(s)) );
    assert(3 == peek(s));
    assert(3 == pop(s));
    assert(2 == peek(s));
    assert(2 == pop(s));
    assert(1 == peek(s));
    assert(1 == pop(s));
    assert((isEmpty(s)));

    printf("OK!\n");
    printf("-----\n");
    // Don't forget to free the memory!
    deleteStack(s);
}

```

Discussione e miglioramenti

Verifica condizioni di overflow/underflow

Possiamo assegnare alla funzione `pop()` la responsabilità di verificare che lo stack non sia vuoto. `pop()` rende un `bool` per indicare il successo o fallimento dell'operazione, e usa un parametro di output (`int *val`) per rendere il valore.

```
// PRECONDIZIONE: stack NON vuoto
int pop(Stack * stack){
    int val = stack->stackArray[stack->top];
    stack->top--;
    return val;
}
```

```
// la responsabilità di verificare che lo stack non sia vuoto è di pop()
bool pop(Stack *stack, int * val){
    if (isEmpty(stack)) {
        return false;
    }

    *val = stack->stackArray[stack->top];
    stack->top--;
    return true;
}
```

Modifiche analoghe vanno applicate a `peek` e `push` per mantenere l'interfaccia coerente.

Stack con capacità dinamica

Abbiamo gestito la dimensione dello stack in modo molto semplice:

```
#define STACK_MAX_SIZE 1000

typedef struct stack {
    int stackArray[STACK_MAX_SIZE];
    int top;
}Stack;
```

Definire la dimensione massima dello stack nell'interfaccia (ad esempio tramite una costante) introduce un limite rigido: tutti gli stack hanno la stessa capacità, fissata a compile-time, e il client non può adattarla alle proprie esigenze. Questo riduce la flessibilità e lega l'uso dell'ADT a una scelta implementativa.

Una soluzione è permettere all'utente di specificare la capacità al momento della creazione dello stack. In questo modo, la funzione `initStack` deve essere modificata per allocare **dinamicamente** anche il vettore che contiene gli elementi, che prima era statico. L'array non è più contenuto direttamente nella struttura, ma viene allocato separatamente in memoria dinamica. Si introduce quindi una **doppia allocazione dinamica**, una per la struttura `Stack` e una per il vettore di dati.

Di conseguenza, anche `deleteStack` deve essere aggiornata per liberare correttamente entrambe le aree di memoria.

- **Struttura dati**

```
typedef struct stack {  
    int *data;  
    int top;  
    int capacity;  
} Stack;
```

- **Creazione e distruzione**

```
#include <stdlib.h>  
  
Stack *initStack(int capacity) {  
    if (capacity <= 0) return NULL;  
  
    Stack *s = malloc(sizeof(Stack));  
    if (s == NULL) return NULL;  
  
    s->data = malloc(capacity * sizeof(int));  
    if (s->data == NULL) {  
        free(s);  
        return NULL;  
    }  
  
    s->top = -1;  
    s->capacity = capacity;  
  
    return s;  
}  
  
void deleteStack(Stack *s) {  
    if (s == NULL) return;  
  
    free(s->data);
```



```
    free(s);  
}
```

- **Conseguenze sul resto del codice**

Poiché la capacità non è più una costante ma un campo della struttura, anche le funzioni che dipendono da essa devono essere aggiornate. In particolare, `isFull` deve confrontare `top` con `capacity - 1`, invece di usare una dimensione fissata a compile-time.

Rendere dinamica la capacità dello stack aumenta la flessibilità, ma introduce una gestione della memoria più complessa e richiede maggiore attenzione nell'implementazione.

- **Esempio d'uso**

```
Stack *s = initStack(50);
```

Dangling pointer

Una funzione come

```
void deleteStack(Stack *s) {  
    free(s);  
}
```

libera correttamente la memoria puntata da `s`, ma **non modifica il puntatore del chiamante**. Di conseguenza, dopo la chiamata, nel `main` la variabile `s` continua a contenere l'indirizzo di una zona di memoria ormai liberata: si ha quindi un **dangling pointer**. Un puntatore dangling è pericoloso perché può essere usato accidentalmente (dal `main`) dopo la `free`, producendo comportamento indefinito.

Per evitare il problema, una prima soluzione è lasciare la responsabilità al chiamante, che deve mettere a `NULL` il puntatore.

```
// main  
  
deleteStack(s);  
s=NULL;
```

Una seconda soluzione, più robusta, è passare alla funzione **l'indirizzo del puntatore**, in modo che la funzione possa azzerarlo direttamente:

```
void deleteStack(Stack **s) {
    if (s == NULL || *s == NULL) return;
    free(*s);
    *s = NULL;
}
```

la chiamata in C è:

```
deleteStack(&s);
```

Se lo stack contiene anche memoria dinamica interna, per esempio un array **data**, allora prima di **free(*s)** bisogna liberare anche quella memoria.

Stack con Array Statico, incapsulamento completo

Nel progetto precedente abbiamo già adottato diverse scelte che rendono naturale il passaggio all'incapsulamento completo: il **main** manipola solo puntatori a **Stack**, la creazione e la distruzione della struttura avvengono tramite funzioni del modulo. Per questo motivo, il passaggio a un ADT con incapsulamento completo è quasi immediato: è sufficiente spostare la definizione completa della struttura nel file **.c** e lasciare nel **.h** solo la dichiarazione del tipo incompleto e delle funzioni dell'interfaccia.

In questo modo, la rappresentazione interna dello stack non è più accessibile al codice client, che può usare la struttura solo attraverso le operazioni fornite dal modulo. La sola vera condizione è che il codice client non acceda più direttamente ai campi della struttura.

Applichiamo anche le principali migliorie discusse, come la capacità dinamica, l'aggiornamento di **deleteStack** e i controlli sulle allocazioni.

Rimane il problema del dangling pointer, di cui abbiamo già discusso. Il codice finale è disponibile sul repository ed è riportato anche di seguito.

module.h

```
// ADT/stack_array_encaps/module.h

/* STACK
 * Implementazione: array dinamico
 * CON incapsulamento completo
 *
 */

#ifndef MODULE_H
```

```

#define MODULE_H
#include <stdbool.h>

typedef struct stack Stack;

// Function prototypes

// Push element to the top of the stack

bool push(Stack * stack, int val);

// Remove the top most element of the stack and store it in val
bool pop(Stack * stack, int * val);

// Return the top most element of the stack and store it in val
bool peek(Stack * stack, int * val);

// Check if the stack is in Underflow state or not
bool isEmpty(Stack * stack);

Stack * initStack(int capacity); //INIT
void deleteStack(Stack * stack); // Free memory

// Check if the stack is in Overflow state or not
bool isFull(Stack * stack);

// FOR DEBUG USE ONLY - REMOVE FROM THE INTERFACE
void show(Stack * stack);
void automaticTest(void);

#endif //MODULE_H

```

module.c

```

// ADT/stack_array_encaps/module.c

#include <stdlib.h>
#include <stdio.h>
#include <assert.h>

```

```

#include "module.h"

struct stack {
    int *data;
    int top;
    int capacity;
};

Stack *initStack(int capacity) {
    if (capacity <= 0) return NULL;

    Stack *s = malloc(sizeof(Stack));
    if (s == NULL) return NULL;

    s->data = malloc(capacity * sizeof(int));
    if (s->data == NULL) {
        free(s);
        return NULL;
    }

    s->top = -1;
    s->capacity = capacity;

    return s;
}

void deleteStack(Stack *s) {
    if (s == NULL) return;

    free(s->data);
    free(s);
}

bool push(Stack * stack, int val){
    if (isFull(stack)) {
        return false;
    }

    stack->top+=1;
    stack->data[stack->top] = val;
    return true;
}

bool pop(Stack *stack, int * val){

```

```

    if (val == NULL || isEmpty(stack)) {
        return false;
    }

    *val = stack->data[stack->top];
    stack->top--;
    return true;
}

bool peek(Stack * stack, int * val){
    if (val == NULL || isEmpty(stack)) {
        return false;
    }

    *val = stack->data[stack->top];
    return true;
}

bool isEmpty(Stack * stack){ return stack->top== -1; }

bool isFull(Stack * stack){ return stack->top == stack->capacity-1; }

// FOR DEBUG USE ONLY - REMOVE FROM THE INTERFACE
void show(Stack * stack){
    int i;
    if (isEmpty( stack)) {
        printf("Empty stack!\n");
        return;
    }
    printf("[TOP] [");
    for (i=stack->top; i>=0; i--)
        printf("%d ",stack->data[i]);
    printf("] [BASE]\n");
}

void automaticTest(void) {
    int value;
    int peekedValue;
    int poppedValue;
    Stack * s;
    s = initStack(10);
    assert(s != NULL);

```

```

printf("Automatic test\n");
printf(" - check empty\n");
assert(isEmpty(s)); //https://en.wikipedia.org/wiki/Assert.h
printf(" - check push, pop, peek\n");
// Push 1
value=1;
assert(push(s, value));
assert( !isEmpty(s));
assert(peek(s, &peekedValue));
assert(value == peekedValue);
assert( !isEmpty(s));
assert(pop(s, &poppedValue));
assert(value == poppedValue);
assert((isEmpty(s)));

// Push 1, 2, 3
assert(push(s, 1));
assert(push(s, 2));
assert(push(s, 3));
printf("->expected: \n[TOP] [3 2 1] [BASE]\n");
printf("->obtained:\n");
show(s);
assert( !isEmpty(s));
assert(peek(s, &peekedValue));
assert(3 == peekedValue);
assert(pop(s, &poppedValue));
assert(3 == poppedValue);
assert(peek(s, &peekedValue));
assert(2 == peekedValue);
assert(pop(s, &poppedValue));
assert(2 == poppedValue);
assert(peek(s, &peekedValue));
assert(1 == peekedValue);
assert(pop(s, &poppedValue));
assert(1 == poppedValue);
assert((isEmpty(s)));

printf("OK!\n");
printf("-----\n");
// Don't forget to free the memory!
deleteStack(s);
}

```

💡 Esercizi

→ **Esercizio.** Implementate ‘da zero’ lo stack basato su array con incapsulamento completo

→ **Esercizio.** Calcolo della media nello stack

Aggiungere allo stack una funzione che calcoli la **media degli elementi contenuti**.

Esistono diverse possibili strategie di implementazione, con diversi compromessi tra semplicità ed efficienza:

- **Compute on demand** La media viene calcolata ogni volta che la funzione è chiamata, iterando su tutti gli elementi dello stack.
 - implementazione semplice
 - costo lineare ($O(n)$) ad ogni chiamata
- **Running mean (media incrementale)**. La media (o la somma) viene aggiornata ogni volta che si esegue una **push** o una **pop**. Il valore è memorizzato nella struttura.
 - accesso immediato alla media ($O(1)$)
 - maggiore complessità: bisogna aggiornare correttamente il valore ad ogni operazione
- **Lazy evaluation**. La media viene calcolata solo alla prima richiesta e poi memorizzata. Quando lo stack cambia, la media viene invalidata e ricalcolata alla successiva richiesta.
 - evita calcoli inutili
 - efficiente se la media è richiesta raramente o mai, o se richiesta spesso ma senza aggiornamenti dello stack.
 - richiede la gestione di uno stato “valido/non valido” (es. flag)

Variante: minimo, massimo...

→ **Esercizio.** Modificare l’implementazione dello stack per supportare l’operazione ‘reverse()’

(COMPLESSO)

Aggiungere un’operazione **reverse()** che inverta l’ordine degli elementi attualmente presenti nello stack. Dopo l’inversione, lo stack deve continuare a funzionare correttamente con le operazioni standard (**push**, **pop**, ecc.).

L’inversione deve essere effettuata **in-place**, cioè **senza utilizzare strutture dati ausiliarie** (come altri stack o array), ma solo un numero limitato di variabili temporanee.

Possibili soluzioni:

- **Soluzione 1:** Utilizzare la ricorsione per rimuovere gli elementi e reinserirli in ordine inverso (eventualmente con una funzione ausiliaria come `pushAtBottom`).
Nota: questa soluzione è semplice ma utilizza implicitamente la call stack.

Traccia

```
void pushAtBottom(Stack* stack, int value) {
    if (isEmpty(stack)) {
        push(stack, value);
        return;
    }

    int temp = pop(stack);
    pushAtBottom(stack, value);
    push(stack, temp);
}

void reverse(Stack* stack) {
    if (isEmpty(stack)) {
        return;
    }

    int temp = pop(stack);
    reverse(stack);
    pushAtBottom(stack, temp);
}
```

- **Soluzione 2:** Modificare l'implementazione dello stack per supportare l'operazione `reverse()`

Nella implementazione standard su array, la base dello stack è sempre indicata (implicitamente) dall'indice 0. Possiamo introdurre un nuovo indice **base** che denota esplicitamente la base dello stack. Se invertiamo il ruolo di **base** e **top**, e adattiamo di conseguenza il funzionamento di `push`, `pop`, `peek`, lo stack “appare” invertito, ma **senza spostare fisicamente gli elementi**: l'operazione può quindi essere molto efficiente.

Per ottenere questo comportamento dobbiamo reinterpretare lo stack come una struttura su array con gestione **circolare** degli indici.

In particolare:

- gli indici devono essere aggiornati in modo **modulare** (quando si supera `capacity - 1`, si torna a 0 e viceversa)
- le condizioni di stack vuoto e pieno non possono più essere espresse con semplici confronti lineari, ma devono tener conto della relazione tra **base** e **top**

- può essere necessario introdurre una nozione di **direzione logica** (esplicita o implicita) per interpretare correttamente l'ordine degli elementi

Di fatto, questa soluzione introduce una logica simile a quella di un **buffer circolare**.

La seconda soluzione è più efficiente (l'inversione è “logica”, non fisica), ma introduce una complessità maggiore e si allontana dall'implementazione classica dello stack.
